



INFORME TÉCNICO

ETP Maestranza



Fecha	
Versión	
Clasificación	
Proyecto	

1. Modelos Creados en Base de Datos

Se crearon cuatro modelos Prisma en la base de datos SQLite (dev.db) del proyecto etp-app. Estos modelos conforman la capa de datos del motor de planificación y almacenan las reglas de producción, las restricciones de capacidad y los resultados del planificador.

1.1 LeadTimeByCode (tabla: lead_time_by_code)

Campo	Tipo / Descripción
id	TEXT — Clave primaria (CUID)
codigo_plazo	TEXT — Tipo de equipo, valores 1 a 17
descripcion_equipo	TEXT — Descripción legible del equipo
proceso	TEXT — Nombre del proceso productivo
duracion_dias	INTEGER — Días hábiles requeridos por el proceso

- Restricción de unicidad compuesta: **(codigo_plazo, proceso)**
- Registros cargados: **187** (17 tipos de equipo × 11 procesos)

1.2 ProcessCapacity (tabla: process_capacity)

Campo	Tipo / Descripción
id	TEXT — Clave primaria (CUID)
proceso	TEXT — Nombre del proceso (UNIQUE)
orden	INTEGER — Orden global de ejecución
capacidad_por_dia	INTEGER — Equipos simultáneos máximos por día

- Registros cargados: **11 procesos**

1.3 OptimizedProcessSchedule (tabla: optimized_process_schedule)

Campo	Tipo / Descripción
id	TEXT — Clave primaria (CUID)
sales_planning_id	TEXT — FK → sales_planning (registro de equipo)
proceso	TEXT — Nombre del proceso
orden	INTEGER — Orden de ejecución
start_date	DATETIME — Fecha/hora de inicio del proceso
end_date	DATETIME — Fecha/hora de fin del proceso

Campo	Tipo / Descripción
duration_days	INTEGER — Duración en días hábiles

- Permite trazabilidad completa: cada equipo tiene un registro por proceso.

1.4 SalesPlanningOptimized (tabla: sales_planning_optimized) — actualizado

Campo	Tipo / Descripción
start_date	DATETIME — Inicio del job completo (primer proceso)
end_date	DATETIME — Fin del job completo (último proceso)
prioridad	INTEGER — Prioridad del equipo (1–10)
codigo_plazo	TEXT — Tipo de equipo (referencia a lead times)

- Campos añadidos en esta versión al modelo existente.
- Almacena el resultado *agregado por equipo* (inicio/fin del job completo).

2. Carga desde Excel (seed_rules.py)

El script `scripts/seed_rules.py` lee el archivo Excel de reglas de planificación y carga los datos en la base de datos mediante UPSERT, garantizando idempotencia en ejecuciones repetidas.

- Archivo fuente: `data/Reglas_Planificación.xlsx`
- Conexión directa a SQLite: `etp-app/dev.db`

Hojas leídas

- **CÓDIGO PLAZO Y DEMORAS EN DÍAS** — 17 tipos de equipo × 11 procesos = 187 registros en `lead_time_by_code`
- **CAPACIDAD POR PROCESO** — 11 procesos con orden y capacidad diaria en `process_capacity`

Procesos cargados

Orden	Proceso	Cap./día
1	INGENIERÍA	2
2	CORTE	3
3	PLEGADO	2
4	ARMADO	2
5	REMATE	2
7	MONTAJE	3
8	HIDRÁULICA	6

Orden	Proceso	Cap./día
10	PINTURA	2
11	TERMINACIONES	2
12	CONTROL DE CALIDAD	1
0	INSPECCIÓN	0 (excluido del planificador)

Tabla 1. Procesos productivos: orden de ejecución y capacidad máxima diaria.

3. Motor CP-SAT (scripts/planner.py)

El planificador utiliza la biblioteca **Google OR-Tools CP-SAT** (ortools v9.15), un solver de programación con restricciones orientado a problemas de scheduling industrial. El modelo formulado es un *Job-Shop con recursos compartidos* y función objetivo de minimización de tardanza ponderada.

3.1 Variables de decisión

```
start[j][p] ∈ [llegada_day, HORIZON] # día hábil de inicio de tarea
end[j][p] = start[j][p] + duration[j][p]
interval[j][p] = IntervalVar(start, duration, end)
```

3.2 Restricciones

- **No-earlier-than:** $start[j][0] \geq \text{workday}(llegada[j])$ — ningún proceso puede iniciar antes de la fecha de llegada del equipo.
- **Precedencia:** $start[j][p] \geq end[j][p-1]$ para $p > 0$ — cada proceso debe esperar la finalización del anterior.
- **Capacidad (Cumulative):** $\sum demand[j][p] \leq \text{capacidad_por_dia}[p]$ en todo momento — no se supera la capacidad simultánea de ningún proceso.

3.3 Filtros de proceso

Un proceso es excluido del modelo si se cumple alguna de las siguientes condiciones:

- $\text{duracion_dias} = 0$ para ese tipo de equipo, O
- $\text{capacidad_por_dia} = 0$, O
- $\text{orden} = 0$ (e.g. INSPECCIÓN)

3.4 Función objetivo

Minimizar la **tardanza total ponderada por prioridad**:

```
min Σ_j ( tardanza[j] × peso[j] )
# donde:
tardanza[j] = max( 0, end[j][last] - due_date[j] )
due_date[j] = llegada[j] + Σ duraciones[j] + atraso[j]
peso[j] = 100 ÷ prioridad[j]
```

prioridad 1 → peso 100 | prioridad 5 → peso 20

3.5 Configuración del solver

- Tiempo límite: **30 segundos**
- Workers paralelos: **4**

3.6 Días hábiles

- Solo **lunes a viernes**. Sábado y domingo excluidos automáticamente.
- Sin feriados en esta fase (Fase 1).
- Referencia: lunes al inicio o antes de la fecha mínima de llegada del lote.
- Conversión date → workday: cuenta días Mon–Fri entre referencia y fecha.
- Conversión workday → date: avanza días hábiles desde la referencia.

4. Uso de Prioridad y Atraso

4.1 Prioridad

- Escala **1–10** (1 = más urgente, 10 = menos urgente).
- Campo **OBLIGATORIO** desde esta versión del sistema.
- Usado como peso en la función objetivo: **peso = 100 ÷ prioridad**
- Un equipo P1 tiene **10×** más peso que uno P10.

4.2 Atraso (buffer)

- Días hábiles de margen tolerados después de la duración nominal.
- Campo **OBLIGATORIO** (valor 0 = sin tolerancia de atraso).
- $\text{due_date} = \text{llegada} + \sum \text{duración_procesos} + \text{atraso}$
- La tardanza se penaliza sólo si $\text{fin} > \text{due_date}$.

4.3 Campos obligatorios en el formulario

Campo	Tipo / Descripción
codigo_plazo	Define los procesos y duraciones del equipo (1–17)
llegada	Fecha de llegada — restricción no-earlier-than del primer proceso
prioridad	Escala 1–10 — peso en la función objetivo
atraso	Días hábiles de buffer (≥ 0 , valor 0 = sin tolerancia)

5. Comandos a Ejecutar

5.1 Setup inicial (solo primera vez)

```
# Instalar dependencias Python
pip3 install ortools openpyxl

# Cargar reglas desde Excel
python3 scripts/seed_rules.py
```

5.2 Servidor de desarrollo

```
cd etp-app
npm run dev

# Abrir: http://localhost:3000
```

5.3 Prisma (si se modifican modelos)

```
# Usar Node.js v22 (requerido por Prisma 7)
export PATH="$HOME/.local/share/fnm/node-versions/v22.22.2/installation/bin:$PATH"
npx prisma generate
```

5.4 Re-cargar reglas de planificación

```
python3 scripts/seed_rules.py [ruta_db_opcional]
```

5.5 Planificar manualmente (sin UI)

```
python3 scripts/planner.py [ruta_db_opcional]
```

6. Reinicio del Servidor

Es necesario reiniciar el servidor Next.js después de cualquier cambio en el schema de Prisma o regeneración del cliente.

- La migración de schema se aplica directamente al SQLite.
- El cliente Prisma debe ser regenerado con **npx prisma generate**.
- El build de producción debe verificarse sin errores antes del deploy.

Comando de reinicio

```
cd etp-app && npm run dev
```

7. Limitaciones Actuales

Las siguientes limitaciones son conocidas en la Fase 1 (MVP). Serán abordadas en fases posteriores del proyecto.

#	Limitación	Descripción
1	Sin feriados	Solo se excluyen sábados y domingos. No se consideran feriados chilenos ni días de parada programada. Impacto: las fechas planificadas pueden no coincidir con el calendario real.
2	Sin talleres / recursos específicos	El modelo asume que un equipo ocupa 1 unidad de capacidad por proceso. No hay modelado de operarios específicos ni máquinas individuales.
3	Capacidad diaria vs. simultáneo	La restricción Cumulative modela días laborales como intervalos enteros. Un equipo con duración = 2 ocupa 2 slots consecutivos.
4	Procesamiento síncrono	El botón "Planificar" bloquea la UI hasta que el solver termina (máx. 30 s). Para flotas grandes se recomienda implementar un job asíncrono (queue + polling).
5	Sin replanificación incremental	Cada ejecución limpia el plan anterior completo. No es posible fijar tareas ya iniciadas y replanificar solo las pendientes.
6	Sin mantenimiento de equipos	No se modela tiempo muerto de proceso entre equipos (setup time, limpieza, calibración).
7	Node.js v22 requerido para Prisma 7	La versión del sistema (v18) no es compatible. Se debe usar fnm con v22.22.2 para todas las operaciones Prisma (generate, migrate, studio).
8	Datos de prueba — migración manual	La migración se aplicó manualmente para preservar los registros existentes. En producción utilizar npx prisma migrate deploy .

— Fin del informe —